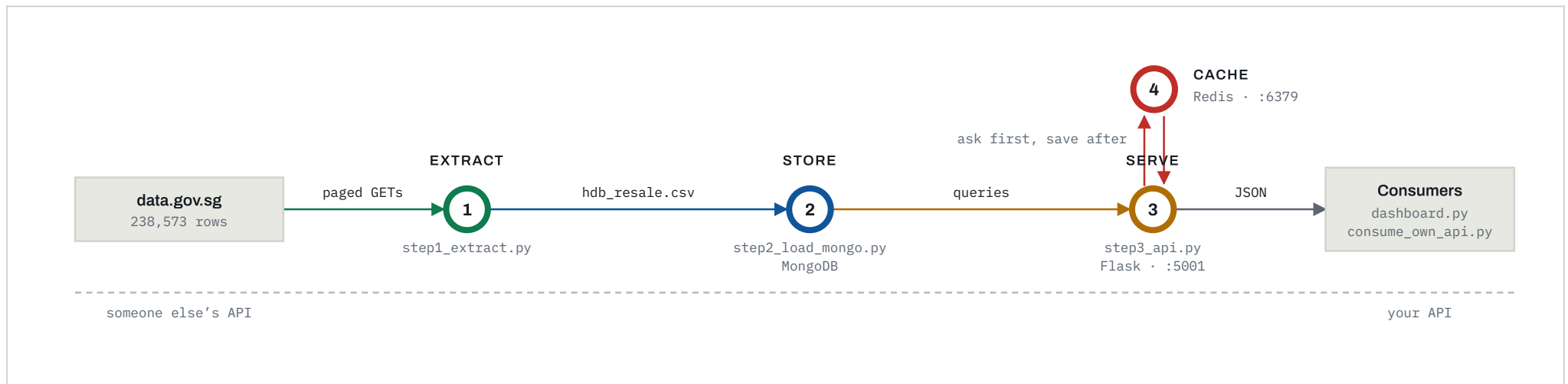


The whole project, and what each command does

Four files carry real Singapore HDB resale data from a government API to a dashboard. Five commands run them.



The flow. Data moves left to right, once. Redis is not a stop on the way — it sits beside the API, which checks it before walking to the database.

THE FOUR FILES

1 **Extract** `src/step1_extract.py`

Pages through the data.gov.sg API and lands the rows as a CSV.

2 **Store** `src/step2_load_mongo.py`

Reads that CSV into MongoDB and indexes it. This is the source of truth.

3 **Serve** `src/step3_api.py`

A Flask API on port 5001 that answers questions about the collection.

4 **Cache** `src/step4_cache_redis.py`

Keeps the expensive answers in Redis for 60 seconds so they are not recomputed.

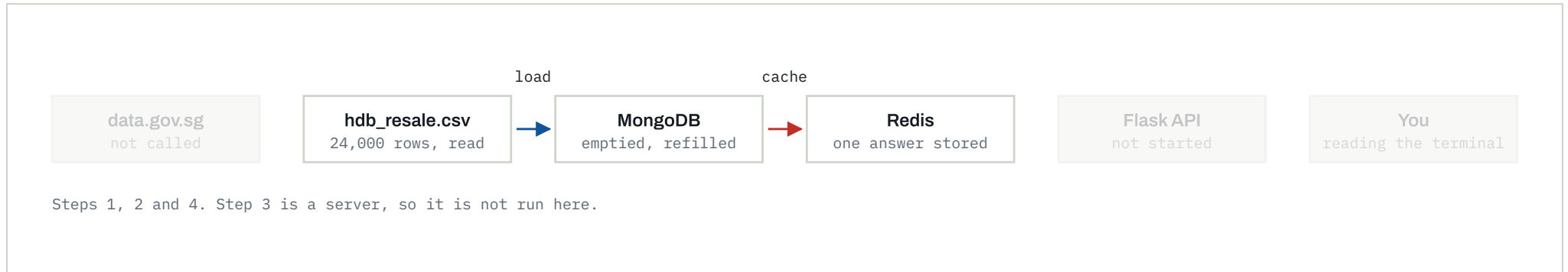
COMMAND BY COMMAND

What happens when you run it

In each diagram below, the highlighted boxes are the parts that command actually touches. Faded boxes are not involved. MongoDB and Redis must already be running.

```
./run.sh pipeline
```

A FEW SECONDS · NO NETWORK



- 1 **Step 1 skips the network.** The CSV ships with the repo, so it reads the landed file and says so.
- 2 **Step 2 rebuilds the collection.** It deletes what was there, inserts every row, and indexes `town` – which is why running it twice is safe.
- 3 **Step 4 asks the same question twice.** The first call computes it in MongoDB and stores it; the second comes straight back from Redis.

```
STEP 2 - Load into MongoDB (system of record)
24,000 documents in resale_flats, 23 towns
```

```
STEP 4 - Cache with Redis (the speed layer)
CACHE MISS overview -> aggregating in MongoDB 34.79 ms
CACHE HIT overview 0.18 ms
```

```
./run.sh pipeline --extract
```

ABOUT 70 SECONDS · NEEDS INTERNET

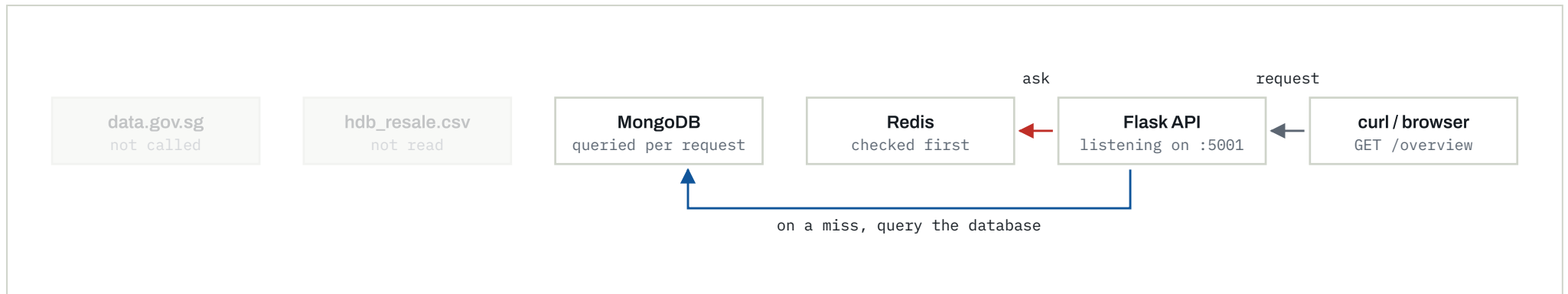


Same as above, with Step 1 going to the live API first.

- 1 **26 requests, 1.2 seconds apart.** The API caps each response, so you page – and the pages are spread across the dataset so the sample covers most towns rather than just the first one.
- 2 **If it fails, it copes.** A 429 means back off and retry; no connection at all means fall back to the CSV from last time and carry on.
- 3 **Then it continues as normal.** The new rows overwrite the CSV, and Steps 2 and 4 run exactly as they do above.

```
./run.sh api
```

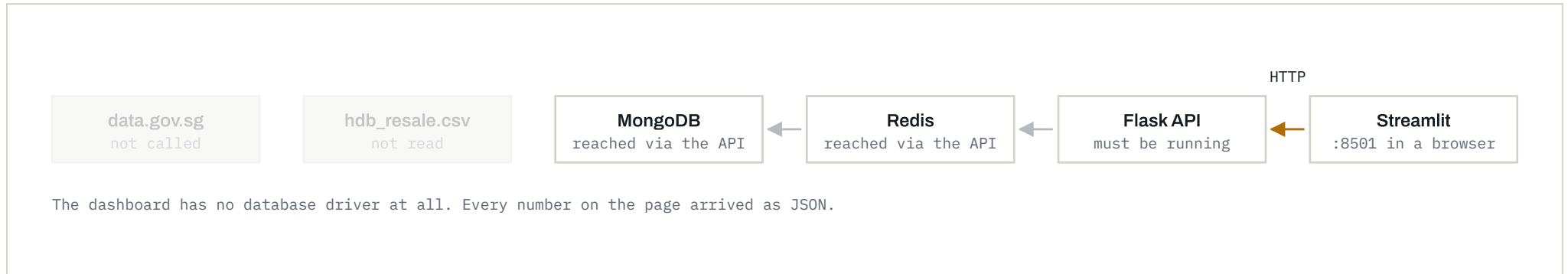
STAYS RUNNING · OWN TERMINAL



- 1 **Nothing is copied.** The API only answers questions about what Step 2 already loaded, so an empty database means empty answers.
- 2 **Add `?cache=true` to compare.** The same route runs with and without Redis, and reports how long it took either way.
- 3 **It blocks until you stop it.** That is what a server does – leave it running and open another terminal.

```
./run.sh dashboard
```

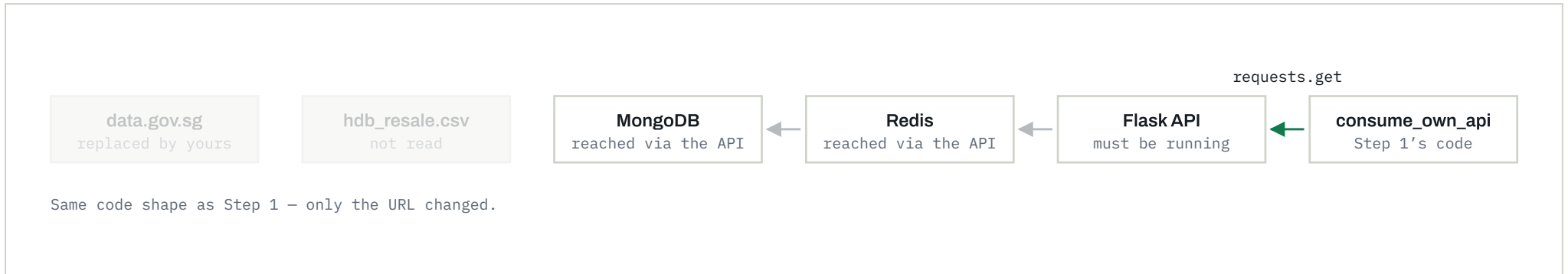
STAYS RUNNING · NEEDS THE API UP



- 1 **It is just another API client.** Every chart and table is an HTTP call to `localhost:5001`.
- 2 **So the API has to be up first.** If it is not, the page says so rather than showing an empty chart.
- 3 **Changing a dropdown re-runs the script.** New selection, new request, new answer.

```
./run.sh consume
```

A FEW SECONDS · NEEDS THE API UP



- 1 **This is Step 1 again.** The same `requests.get(...)` , pointed at `localhost:5001` instead of `data.gov.sg`.
- 2 **The code cannot tell the difference.** To it, your API is just a source of JSON like any other.
- 3 **That closes the loop.** Extraction is not a one-time first step – it is a role any system can play, depending on which side of the request you are on.

REFERENCE

The five commands

```
./run.sh pipeline           # steps 1, 2, 4 – from the CSV in the repo
./run.sh pipeline --extract  # same, but re-pull from data.gov.sg first
./run.sh api                # step 3 – serve on :5001      (own terminal)
./run.sh dashboard          # Streamlit on :8501           (own terminal)
./run.sh consume            # extract from your own API
```

Start MongoDB and Redis before any of them. Check with `curl localhost:5001/health` once the API is up.

Contains information from **Resale Flat Prices**, accessed 21 August 2026 from data.gov.sg, under the Singapore Open Data Licence version 1.0. No API key needed.